
THE DEFINITIVE GUIDE

Top Ten AI Design Patterns

*Lessons Learned from Real AI
Implementations*

01 Single API Call

03 Parallel

05 Map-Reduce

07 Fallback

09 Plan-Execute

02 Chain

04 Route

06 Evaluate-Refine

08 Tool Loop (Agent)

10 Human-in-Loop

Pattern #1: Single API Call

Best for: Simple, self-contained tasks

- Extract contact info from an email
- Summarize a paragraph
- Classify sentiment

WHEN TO USE:

If it fits in one request with no decisions, don't overcomplicate it.

PYTHON EXAMPLE

```
response = client.messages.create(  
    model="claude-sonnet-4-20250514",  
    messages=[{"role": "user",  
                "content": "Extract email from: Contact  
John at john@example.com"}]  
)
```



Pattern #2: Chain

Multiple LLM calls in sequence

Best for: Multi-step tasks with known order

→ Classify support ticket → Fetch docs → Draft response

WHY IT WORKS:

Predictable. Debuggable. Easy to monitor.

PYTHON EXAMPLE

```
# Step 1: Classify
category = llm.call("Classify this ticket: {ticket}")

# Step 2: Fetch context
docs = database.get_docs(category)

# Step 3: Generate response
response = llm.call(f"Draft response using: {docs}")
```



GET HIRED

The only bundle you'll ever need.



Stop scattered prep. Start structured learning.

13 modules · 645 questions · built for AI engineers



Link of Bundle in caption

Pattern #3: Parallel

Run independent tasks simultaneously. Combine results at the end.

Best for: Steps that don't depend on each other

- Generate images while generating text
- Process 10 documents at once

THE MATH:

N tasks in parallel = N times faster

PYTHON EXAMPLE

```
import asyncio

tasks = [
    llm.call_async(f"Summarize doc {i}")
    for i in range(10)
]
results = await asyncio.gather(*tasks)
```



Pattern #4: Route

Classify input first. Branch to specialized handlers.

Best for: Different inputs need different treatment

- Complaints → Empathetic response
- Sales inquiries → Forward to sales
- Spam → Discard

ARCHITECTURE:

One router. Many expert paths.

PYTHON EXAMPLE

```
intent = llm.call("Classify intent: {message}")

handlers = {
    "complaint": handle_complaint,
    "sales": forward_to_sales,
    "spam": discard
}
handlers[intent](message)
```



Pattern #5: Map-Reduce

Process many items independently. Synthesize into one output.

Best for: Batch processing or exceeding context limits

- Summarize 50 documents into one report
- Analyze 100 customer reviews

STRATEGY:

Divide. Conquer. Combine.

PYTHON EXAMPLE

```
# Map: Process each document
summaries = [
    llm.call(f"Summarize: {doc}")
    for doc in documents
]

# Reduce: Combine all summaries
final = llm.call(f"Synthesize these: {summaries}")
```



Pattern #6: Evaluate-Refine

Generate → Check against criteria → Improve → Loop until good enough.

Best for: Clear quality standards exist

- Code that must pass tests
- JSON that must validate
- Copy that hits word count

PHILOSOPHY:

Quality through iteration, not luck.

PYTHON EXAMPLE

```
for attempt in range(max_iterations):
    output = llm.call(prompt)

    if meets_criteria(output):
        return output

    prompt = f"Improve this: {output}. Issues: {get_issues(output)}"
```



Pattern #7: Fallback

Try fast, cheap model first. Escalate to powerful model only if needed.

Best for: Most requests are easy, some are hard

→ Haiku handles 80% of queries

→ Sonnet handles the rest

RESULT:

Same quality. Fraction of the cost.

PYTHON EXAMPLE

```
response = llm.call(prompt, model="haiku")

if confidence_score(response) < threshold:
    response = llm.call(prompt, model="sonnet")

return response
```



Pattern #8: Tool Loop (Agent)

Model decides what to do next. You execute. Feed results back. Repeat until done.

Best for: Unknown steps in advance

- Research tasks
- Debugging sessions
- Open-ended exploration

TRADE-OFF:

Capability + unpredictability

PYTHON EXAMPLE

```
while not task_complete:
    decision = llm.call(f"Next action? Context:
{context}")

    result = execute_tool(decision.tool, decision.args)
    context.append(result)
```



Pattern #9: Plan-Execute

Make a plan first. Execute each step. Replan when reality changes.

Best for: Complex tasks benefiting from upfront thinking

- Multi-file code changes
- Project migrations
- Dependency analysis

MANTRA:

Think first. Act second.

PYTHON EXAMPLE

```
plan = llm.call(f"Create plan for: {task}")

for step in plan.steps:
    result = execute(step)

    if result.needs_replan:
        plan = llm.call(f"Replan given: {result}")
```



Pattern #10: Human-in-the-Loop

Pause for human approval at critical decision points.

Best for: High stakes or irreversible actions

- Auto-approve low-risk actions
- Require review for payments/deletions

BALANCE:

AI proposes. Human verifies.

PYTHON EXAMPLE

```
action = llm.call(f"Suggest action for: {input}")

if action.risk_level > threshold:
    action = await request_human_approval(action)

execute(action)
```



Pick the Right Pattern

QUICK GUIDE

- Simple task? **Single call**
- Known steps? **Chain**
- High stakes? **Human-in-the-loop**
- Unknown path? **Agent**

Start simple. Add complexity only when needed.

What pattern are you using?

Drop a comment below! 📌



GET HIRED

The only bundle you'll ever need.



Stop scattered prep. Start structured learning.

13 modules · 645 questions · built for AI engineers



Link of Bundle in caption

Want more content like this?



Tap that follow button and
stay in the loop!



Like



Comment



Share



Save